

```
#include <iostream> // this line includes input and output stream library
```

```
#include <vector> // vector library for dynamic array
```

```
#include <chrono> // to measure execution time of the program
```

```
using namespace std; // allows user to use standard library objects
```

```
using namespace std::chrono; // direct use of chrono functions
```

```
// Linear Search Function
```

```
int linearSearch(vector<int> arr, int key)
```

```
{
```

```
    for (int i = 0; i < arr.size(); i++)
```

```
    {
```

```
        if (arr[i] == key)
```

```
            return i;
```

```
    }
```

```
    return -1; // means it not found
```

```
}
```

```
// Binary Search Function
```

```
// it works on sorted array only
```

```
int binarySearch(vector<int> arr, int key)
```

```
{
```

```
    int low = 0; //lowest index
```

```

int high = arr.size() - 1; //highest index

while (low <= high) //divides the array every time until the search range
becomes empty
{
    int mid = low + (high - low) / 2; //to find the mid

    if (arr[mid] == key)
        return mid;
    else if (arr[mid] < key) // if mid is less than key then key is in rightside
        low = mid + 1;
    else
        high = mid - 1; // if not leftside
}

return -1;
}

int main()
{
    int n = 100000; // creates array size of 100,000
    vector<int> arr(n); // creates a vector

    // Creating a sorted array
    for (int i = 0; i < n; i++)

```

```
arr[i] = i + 1;
```

```
int key;
```

```
cout << "Enter element to search: ";
```

```
cin >> key;
```

```
int index;
```

```
// Linear Search Timing
```

```
auto start = high_resolution_clock::now();
```

```
index = linearSearch(arr, key); // calling function
```

```
auto stop = high_resolution_clock::now();
```

```
cout << "\nLinear Search\n";
```

```
if (index != -1)
```

```
    cout << "Element found at index " << index << endl;
```

```
else
```

```
    cout << "Element not found\n";
```

```
cout << "Time Taken : "
```

```
    << duration_cast<microseconds>(stop - start).count()
```

```
    << " microseconds\n";
```

```
// Binary Search Timing
```

```
start = high_resolution_clock::now();
```

```
index = binarySearch(arr, key); // calling binary function
stop = high_resolution_clock::now();

cout << "\nBinary Search\n";
if (index != -1)
    cout << "Element found at index " << index << endl;
else
    cout << "Element not found\n";

cout << "Time Taken : "
    << duration_cast<microseconds>(stop - start).count()
    << " microseconds\n";

return 0;
}
```